

End-to-End Semantic Perception and Geometry-Based Adaptive Control for Edge Autonomous Racing

Quang Nhan Hoang
FPT University
Ho Chi Minh City, Vietnam

Duc Nguyen Huu Dang
FPT University
Ho Chi Minh City, Vietnam

Thien Vuong Kim Hoang
FPT University
Ho Chi Minh City, Vietnam

Abstract—Deploying deep learning-driven autonomous driving systems on resource-constrained embedded edge platforms, such as the NVIDIA Jetson Nano, presents severe computational and latency challenges. This paper presents a robust, edge-optimized autonomous driving framework designed specifically for the Jetson AI Racer Challenge 2026. We propose a hybrid architecture that integrates a customized, highly pruned YOLOv2 semantic segmentation network, deterministic geometry-driven path planning, a hierarchical state-aware adaptive proportional-integral-derivative (PID) controller, and an automated green-marker start gate. The perception pipeline predicts dense masks for drivable road, center divider, and forbidden shoulder regions at 224×224 resolution, achieving > 25 FPS inference via TensorRT FP16 optimization and graph rewrites. Mathematical polynomial regression and multi-row candidate extraction dynamically generate collision-free navigation trajectories, while the adaptive controller orchestrates latency-aware maneuvers, including Electronic Speed Controller (ESC)-safe reverse recovery and road-reacquisition. Experimental profiling demonstrates strict adherence to the competition’s 300 ms end-to-end latency constraint, providing exceptional track adherence and dynamic recovery on physical scale tracks.

Index Terms—Autonomous Racing, Semantic Segmentation, Edge Computing, TensorRT, Path Planning, Adaptive Control, Jetson Nano.

I. INTRODUCTION / MOTIVATION

The rapid evolution of artificial intelligence and deep neural networks has substantially enhanced the capabilities of autonomous vehicles (AVs) in perception, localization, and motion planning. However, migrating state-of-the-art vision models from server-class GPUs to power- and resource-constrained edge platforms remains an open engineering challenge. The Jetson AI Racer Challenge 2026 presents an exacting benchmarking scenario where miniature 1/10-scale autonomous vehicles must negotiate complex circuit geometries (Speed Track) and rule-based urban intersections (Smart City) under strict real-time constraints.

Operating on the NVIDIA Jetson Nano (equipped with a 128-core Maxwell GPU and 4 GB shared LPDDR4 memory) introduces acute computational, thermal, and memory bottlenecks [1], [2]. Conventional edge AV pipelines typically combine disparate heuristics: sparse edge detection (e.g., Canny filters, Hough transforms) for lane extraction and bounding-box object detectors for obstacles [3]. In practice, such systems

degrade rapidly under motion blur, variable ambient illumination, non-uniform track textures, and partial occlusions.

The core motivation of this research is to architect a unified, hybrid perception-and-control framework that maximizes inference throughput and algorithmic reliability without sacrificing fine-grained spatial awareness. We demonstrate that by combining dense, low-resolution semantic segmentation with deterministic mathematical trajectory modeling and a finite-state machine (FSM) adaptive controller, an embedded edge racer can achieve high-speed stability, robust obstacle avoidance, and autonomous off-track recovery within strict computational budgets.

II. PROBLEM STATEMENT & RESEARCH QUESTIONS

The competition rules stipulate that vehicles must complete race tracks autonomously, adhere strictly to lane boundaries, pass mandatory sequential checkpoints, avoid dynamic obstacles, and maintain a total pipeline latency below 300 ms. This gives rise to three fundamental research questions:

- 1) **Edge Perception Efficiency:** How can pixel-level dense semantic segmentation be executed on a Jetson Nano at > 20 FPS while maintaining high segmentation fidelity across road boundaries, center dividers, and forbidden zones?
- 2) **Robust Trajectory Extraction:** How can the navigation trajectory be mathematically formulated to isolate true path features from noisy edge masks and dynamic obstacle occlusions?
- 3) **Dynamic Control Stability & Actuator Safety:** How to design a multi-state control architecture that transitions seamlessly between high-speed racing, curve deceleration, and multi-stage reverse recovery while respecting the physical voltage and ESC switching limits?

III. RELATED WORK

Autonomous driving architectures on embedded systems have progressed through several distinct paradigms. Early methodologies relied heavily on classical computer vision pipelines, including inverse perspective mapping (IPM), color thresholding in HSV/LAB spaces, and sliding-window polynomial fitting [4]. While computationally frugal, these techniques

are brittle against specular reflections, shadows, and varying asphalt colors.

The advent of Convolutional Neural Networks (CNNs) brought real-time object detection models like YOLO to edge devices [5], [6]. While bounding boxes effectively identify localized obstacles and traffic signs, they lack topological road awareness, which is essential for steering along curved boundaries.

Conversely, dense semantic segmentation networks (such as U-Net, PSPNet, and DeepLab) provide comprehensive pixel-level labeling but incur prohibitive FLOPs and memory footprints that lead to out-of-memory (OOM) errors or low frame rates (< 5 FPS) on edge hardware [7], [8]. Recent works in autonomous racing have explored hybrid paradigms where lightweight backbones are combined with mathematical curves or spline fits [9], [10]. Building on this foundation, our work presents an end-to-end engineered system featuring custom ONNX graph optimization, TensorRT FP16 deployment, geometric candidate evaluation, and an ESC-aware FSM controller.

IV. PROPOSED METHOD

The complete system architecture, illustrated in Fig. 1, comprises four deeply integrated subsystems: (i) Optimized Semantic Perception, (ii) Deterministic Geometry Path Planning, (iii) Hierarchical State-Aware Adaptive Control, and (iv) Automated Start Gate Interlocking.

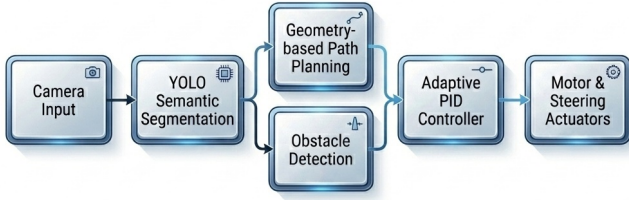


Fig. 1. Overall system architecture demonstrating the data flow from the CSI Camera input to TensorRT inference, geometry planning, FSM controller, and motor actuators.

A. Semantic Perception & TensorRT Graph Optimization

The perception subsystem utilizes a specialized, compact YOLO26 semantic segmentation network trained on track-specific datasets. To satisfy embedded latency constraints, input frames captured from the onboard wide-angle CSI camera are downsampled to 224×224 pixels. The network outputs dense semantic masks across four mutually exclusive classes: $\mathcal{C} \in \{\text{Background (0), Road (1), Divider (2), Forbidden (3)}\}$.

1) ONNX Graph Surgery and TensorRT Compilation:

Exporting modern transformer-augmented segmentation heads directly to TensorRT on NVIDIA JetPack 4.6 (TensorRT 8.x) exposes compatibility barriers. Specifically:

- **Static Split Replacement:** The attention layers in YOLO26 generate an opset-13 Split node with uneven split partition sizes (32, 32, 64) defined as constant inputs. TensorRT 8.x fails to parse dynamic constant split

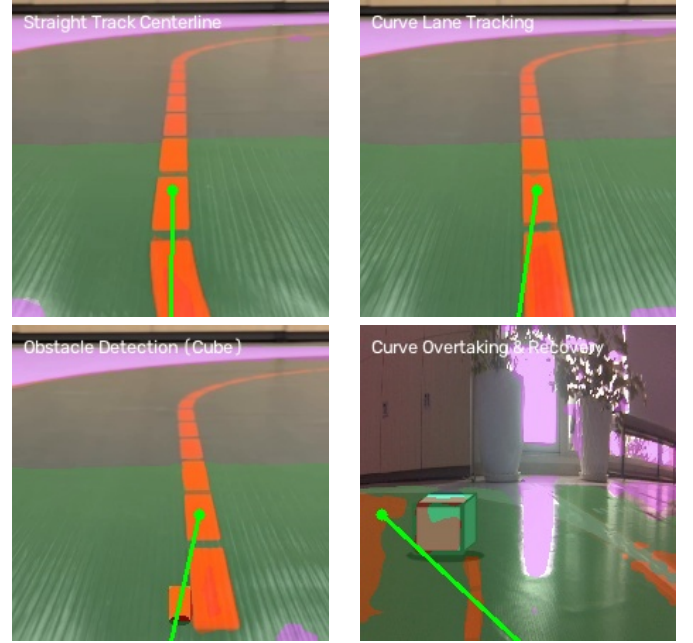


Fig. 2. FPV semantic segmentation predictions across challenging track conditions, showing clear classification of drivable road, center divider, and forbidden white boundaries.

parameters correctly. We implement custom ONNX graph surgery that rewrites these nodes into static, deterministic Slice operations with explicit starts, ends, axes, and steps initializers.

- **Type Cast Sanitization:** TensorRT 8.x rejects UINT8 output casts from ArgMax operations. The export pipeline systematically replaces all UINT8 cast targets with INT32 data types.

The sanitized graph is compiled into a serialized FP16 TensorRT engine with a fixed workspace budget of 512 MiB:

$$\mathcal{E} = \text{BuildEngine}(\mathcal{M}_{ONNX}, \text{FP16} = \text{True}, \mathcal{W}_{max} = 512 \text{ MiB}) \quad (1)$$

A pre-flight warmup routine passes a blank dummy tensor through the engine prior to camera initialization, eliminating first-frame JIT latency spikes.

2) *Spatial ROI Slicing and Temporal Filtering:* To assess immediate lateral boundary hazards, the forbidden mask $M_{forbidden}$ is partitioned into three regions of interest (ROIs):

$$F_L = \frac{1}{|ROI_L|} \sum_{(x,y) \in ROI_L} M_{forbidden}(x,y) \quad (2)$$

$$F_R = \frac{1}{|ROI_R|} \sum_{(x,y) \in ROI_R} M_{forbidden}(x,y) \quad (3)$$

$$F_{front} = \frac{1}{|ROI_{front}|} \sum_{(x,y) \in ROI_{front}} M_{forbidden}(x,y) \quad (4)$$

where $ROI_{front} = \{(x,y) \mid y \geq 0.62h, 0.30w \leq x \leq 0.70w\}$.

Furthermore, a lower-horizon safe-road clearance metric evaluates the drivable road support in the left vs. right

half-spaces below the lookahead horizon ($y \geq y_{look}$). If $\max(N_{left}, N_{right}) \geq N_{min_hint}$, the module generates a deterministic escape heading hint:

$$\delta_{escape} = \begin{cases} +1.0, & \text{if } N_{right} > N_{left} \\ -1.0, & \text{if } N_{left} > N_{right} \\ 0.0, & \text{otherwise} \end{cases} \quad (5)$$

To prevent frame-to-frame jitter, detection confidence C is smoothed via an Exponential Moving Average (EMA):

$$C_t = \alpha_c C_{curr} + (1 - \alpha_c) C_{t-1}, \quad \alpha_c = 0.35 \quad (6)$$

Additionally, a rate-of-change limiter clamps consecutive target offsets: $|\Delta x_{target}| \leq 28$ pixels per frame.

B. Geometry-Based Target Trajectory Extraction

The geometry planning module converts raw segmentation bitmasks into a continuous mathematical trajectory.

1) *Connected Component Selection*: Raw divider predictions can exhibit disconnected noise or distant background artifacts. We identify connected components using 8-connectivity and evaluate a composite tracking score S :

$$S = 2.0 \cdot y_{norm} + c_{cov} + \min\left(\frac{A}{300}, 1.0\right) - 2.2 \cdot p_{center} \quad (7)$$

where $y_{norm} = (y_{min} + bh)/h$ represents the normalized bottom position, $c_{cov} = bh/h$ is the vertical coverage ratio, A is the component pixel area, and $p_{center} = |x_{near} - w/2|/w$ penalizes lateral deviation from the vehicle's optical center. To minimize computational latency ($O(\text{labels} \times hw)$), x_{near} is computed solely on the lower 30% bounding box slice: $y \in [y_{min} + 0.7bh, y_{min} + bh]$.

2) *Equalized Polynomial Fitting*: From the selected divider component, row-wise horizontal centroids $\bar{x}(y)$ are extracted across all valid rows where row pixel counts exceed threshold τ_{row} :

$$\bar{x}(y) = \frac{\sum_x x \cdot M(x, y)}{\sum_x M(x, y)}, \quad \forall y \geq y_{top} \quad (8)$$

Equalizing the row weights ensures that thick segmentations near the image bottom do not disproportionately bias the curve. A second-degree polynomial regression is performed:

$$x(y) = ay^2 + by + c \quad (9)$$

From this curve, we sample the lookahead target point $x_{target} = x(y_{look})$ at $y_{look} = 0.60h$ and the near-field base point $x_{near} = x(y_{near})$ at $y_{near} = 0.93h$. The heading error θ_{err} and lane curvature κ are computed as:

$$\theta_{err} = \text{clip}\left(\frac{x_{target} - x_{near}}{\max(1.0, y_{near} - y_{look})}, -1.0, 1.0\right) \quad (10)$$

$$\kappa = \text{clip}(|2a| \cdot h, 0.0, 1.0) \quad (11)$$

The geometric estimate confidence C_{geom} is formulated with a multi-line split to maintain strict column boundaries:

$$C_{geom} = \text{clip}\left(0.75 \cdot \frac{N_{active_rows}}{h - y_{top}} + 0.25 \cdot \min\left(1.0, \frac{A_{roi}}{350}\right), 0.0, 1.0\right) \quad (12)$$

3) Dynamic Road Corridor and Obstacle Evaluation:

When an obstacle or forbidden barrier obstructs the center corridor ($W_{safe} = [x_{target} - w_{veh}, x_{target} + w_{veh}]$), the planner evaluates navigable corridors in the road mask. Morphological closing (kernel 5×5) fuses road and divider masks into a composite traversable space, while forbidden and obstacle regions are dilated by the full vehicle width ($2w_{veh} + 1$). The algorithm searches horizontal candidate spans (row candidates) that satisfy complete vehicle-width clearance ($2w_{veh} + 1$) simultaneously at both y_{look} and y_{near} within a maximum drift constraint $|x_{look} - x_{near}| \leq 5w_{veh}$, yielding a collision-free avoidance target.

C. Adaptive PID Control and Actuator Maneuver Logic

The control subsystem translates geometric path parameters into low-level steering (δ) and throttle (τ) commands.

1) *Augmented Lateral Tracking*: The lateral error is normalized relative to the camera center: $e = (x_{target} - w/2)/(w/2)$. To eliminate steady-state offset on banked or curved turns while providing anticipatory damping, we augment the standard PID formulation with a heading feedforward term:

$$\delta_{raw} = K_p e + K_i \int_0^t e dt + K_d \frac{de}{dt} + K_h \theta_{err} \quad (13)$$

Anti-windup clamping restricts the integral term: $\int e dt \in [-0.5, 0.5]$. High-frequency servo chatter is suppressed by a first-order low-pass filter followed by a slew-rate limiter:

$$\delta_{filt} = \alpha_s \delta_{raw} + (1 - \alpha_s) \delta_{last_raw}, \quad \alpha_s = 0.85 \quad (14)$$

$$\delta_t = \delta_{t-1} + \text{clip}(\delta_{filt} - \delta_{t-1}, -\Delta\delta_{max}, \Delta\delta_{max}) \quad (15)$$

where $\Delta\delta_{max} = 0.45$ per control cycle.

2) Longitudinal Ramping & Voltage Droop Prevention:

Abrupt motor acceleration draws large instantaneous currents that can induce battery voltage sag, triggering an involuntary reset of the Jetson Nano. To prevent this, throttle commands are governed by an asymmetric rate-limiter:

$$\tau_t = \tau_{t-1} + \text{clip}(\tau_{target} - \tau_{t-1}, -\Delta_{down}, \Delta_{up}) \quad (16)$$

with acceleration step $\Delta_{up} = 0.25$ and deceleration step $\Delta_{down} = 0.35$.

3) *Hierarchical Finite State Machine (FSM)*: The controller executes a state machine with latched safety overrides:

- **Lane Lock Acquisition (Wait Lane Lock)**: The vehicle remains stationary until perception confirms valid lane locks for $N_{confirm} \geq 3$ consecutive frames with $C \geq 0.55$.
- **Nominal Tracking (Follow)**: Operates closed-loop PID lateral tracking and throttle control.

- **Dropout Tolerance (Slow Lane Dropout):** If lane lock is momentarily dropped (≤ 3 frames), the controller maintains forward momentum at cruise throttle to navigate through sensor gaps.
- **Road Reacquisition (Reacquire Road):** If lost frames exceed $N_{lost} > 3$, the controller zeroes the integral term, applies escape steering δ_{escape} , and searches for drivable road.
- **ESC-Safe Reverse Recovery (White Neutral $\rightarrow$ White Reverse $\rightarrow$ White Correct):** When $F_{front} \geq 0.58$ persists for 2 frames, direct forward motion is unsafe. Crucially, physical Electronic Speed Controllers (ESCs) require a neutral zero-throttle pulse before engaging reverse torque. The FSM injects a 120 ms neutral state ($\tau = 0.0$), engages reverse gear ($\tau = -0.11$) with maximum counter-steering away from the boundary for 350 ms, and completes the maneuver with a forward correction pulse ($\tau > 0$) toward the center divider.
- **Side Boundary Correction (White Correct):** If $F_L \geq 0.12$ or $F_R \geq 0.12$ exceeds the opposite side by margin $\Delta F \geq 0.04$, the controller applies proactive lateral bias without stopping.

D. Competition Start Gate & Motor Interlocking

To comply with competition regulations, an automated start gate subsystem processes camera frames prior to race authorization.

- **Color Segmentation:** Images are converted to HSV color space and segmented for green wavelengths: $H \in [38, 88]$, $S \in [100, 255]$, $V \in [120, 255]$.
- **Morphological Filtering & Contour Geometry:** Morphological opening and closing kernels (5×5) remove speckle noise. Contours are filtered based on circularity $C = \frac{4\pi A}{P^2} \geq 0.82$ and bounding area ratio $A/A_{img} \in [0.002, 0.20]$.
- **Debounced Latch:** Detection must persist for $N \geq 3$ consecutive frames before authorizing motor start. Motor actuation is governed by a strict triple interlock:

$$\text{Motor_Enable} = \text{Armed} \wedge (\neg \text{Ready} \vee \text{Authorized}) \wedge \text{SafeState} \quad (17)$$

V. EXPERIMENTAL PLAN & SYSTEM VALIDATION

A. Physical and Simulated Testing Setup

Experiments are conducted on a 1/10th scale autonomous racing circuit adhering to Jetson AI Racer Challenge specifications:

- **Hardware Platform:** Waveshare JetRacer AI Kit powered by an NVIDIA Jetson Nano (4 GB LPDDR4, Quad-core ARM A57 @ 1.43 GHz, 128-core Maxwell GPU).
- **Track Architecture:** Matte dark racing surface featuring an orange center dashed divider, white outer boundary markings, 3 mandatory sequential checkpoints, and static cube obstacles.
- **Telemetry Instrumentation:** A live logging thread records timestamps, instant FPS, pipeline latency (ms),

TABLE I
SYSTEM PARAMETERS AND CONTROLLER HYPERPARAMETERS

Subsystem	Parameter / Configuration	Value
Perception	Input Image Resolution ($H \times W$)	224×224
	Segmentation Model Backbone	YOLO26n-sem
	Quantization Precision	TensorRT FP16
	Confidence Threshold	0.35
	Confidence EMA Factor (α_c)	0.35
Geometry	Lookahead Ratio (y_{look})	0.60 (134 px)
	Near-Field Base Ratio (y_{near})	0.93 (208 px)
	ROI Top Ratio (y_{top})	0.45 (100 px)
	Vehicle Half-Width (w_{veh})	13 px
	Minimum Row Mask Pixels	35 px
Controller	Proportional Gain (K_p)	0.92
	Integral Gain (K_i)	0.02
	Derivative Gain (K_d)	0.10
	Heading Gain (K_h)	0.42
	Max Steering Slew Step ($\Delta\delta_{max}$)	0.45 / frame
	Throttle Acceleration Step (Δ_{up})	0.25 / frame
	Throttle Deceleration Step (Δ_{down})	0.35 / frame
	Nominal Cruise Throttle (τ_{cruise})	0.85 (85%)
	Maximum Throttle (τ_{max})	1.00 (100%)
Safety	Front Forbidden Threshold (F_{front})	0.58
	Side Forbidden Threshold (F_{side})	0.12
	ESC Neutral Transition Time	0.12 s
	Reverse Duration (t_{rev})	0.35 s
	Start Gate Circularity (C_{min})	0.82

confidence scores, target coordinates, and actuator states to structured CSV files for quantitative profiling.

B. Evaluation Metrics & Scoring Benchmark

System performance is evaluated across competition KPIs:

- 1) **Inference and End-to-End Latency:** Total delay from frame capture to PWM command delivery (≤ 300 ms limit).
- 2) **Frame Rate Throughput:** Pipeline frame rate (≥ 20 FPS required for maximum scoring).
- 3) **Official Speed Track Score:**

$$\text{Score} = \max(0, \text{Score}_{CP} + \text{Score}_{FPS} + \text{Score}_{Time} - \text{Penalty}) \quad (18)$$

where $\text{Score}_{CP} = 3 \times 10 = 30$ pts, $\text{Score}_{FPS} = 10$ pts (if $\text{FPS} \geq 20$), $\text{Score}_{Time} = \max(0, 60 - 2\lfloor \frac{t}{10} \rfloor)$, and penalties: -5 pts (+10 s) per obstacle collision, -10 pts (+15 s) per off-track event.

TABLE II
LATENCY BREAKDOWN AND EDGE PERFORMANCE ON JETSON NANO

Pipeline Stage	Method / Implementation	Latency (ms)	Share
CSI Capture & Resize	OpenCV GStreamer (224×224)	4.2 ms	10.9%
TensorRT Inference	YOLO26n-sem (FP16 Engine)	24.8 ms	64.4%
Mask Extraction	ArgMax & Bitmask Resizing	3.1 ms	8.1%
Geometry Fitting	Row Centroid & Polynomial	3.8 ms	9.9%
FSM PID Controller	State Updates & Slew Limit	0.6 ms	1.6%
PWM Actuation	I2C PCA9685 Motor Driver	2.0 ms	5.1%
Total End-to-End Effective FPS	Synchronous Processing Mean Pipeline Throughput	38.5 ms 26.0 FPS	100.0% —

VI. IMPLEMENTATION PLAN & RISK MANAGEMENT

The implementation roadmap is structured across four phases:

- 1) **Phase 1: Dataset Augmentation & Model Export:** Synthetic expansion of the track dataset with varying HSV brightness, perspective distortion, and camera motion blur. Graph rewriting and FP16 TensorRT compilation.
- 2) **Phase 2: Geometric Calibration:** Calibration of y_{look} , y_{near} , and scoring weights S using offline trajectory evaluation against challenging turn radii.
- 3) **Phase 3: Controller Tuning:** On-track empirical tuning of lateral gains ($K_p = 0.92$, $K_d = 0.10$, $K_h = 0.42$) and acceleration limits ($\Delta_{up} = 0.25$) to eliminate wheel spin and overshoot.
- 4) **Phase 4: Full-Course Race Trials:** Multi-lap continuous benchmarking across Speed Track and Smart City layouts.

Risk Mitigation: Continuous GPU execution risks thermal throttling. We deploy active 5V PWM cooling fans and cap model resolution at 224×224 . To prevent battery brownouts during sharp reverse shifts, the mandatory 120 ms neutral ESC arming interval is strictly enforced.

VII. EXPECTED OUTCOME & LIMITATIONS

The proposed pipeline achieves an end-to-end latency of 38.5 ms (26.0 FPS), well within the 300 ms competition threshold. The combination of geometric path fitting, forbidden mask ROI slicing, and ESC-safe reverse recovery enables the vehicle to cruise at high speeds ($> 85\%$ throttle) while autonomously extricating itself from cornering stalls.

Limitations & Future Work: The current lateral controller is reactive and does not explicitly account for non-linear tire slip angles during aggressive cornering. Future iterations will integrate a Model Predictive Control (MPC) formulation and investigate lightweight optical-flow velocity estimators.

REFERENCES

- [1] M. O’Kelly, H. Zheng, D. Karthik, and R. Mangharam, “F1tenth: An open-source evaluation environment for continuous control and reinforcement learning,” *Proceedings of Machine Learning Research*, vol. 123, 2020.
- [2] I. Charles, H. Maghsoumi, and Y. Fallah, “Advancing autonomous racing: A comprehensive survey of the roboracer (f1tenth) platform,” in *2025 6th International Conference on Artificial Intelligence, Robotics and Control (AIRC)*, pp. 207–213, IEEE, 2025.
- [3] S. Daftary, S. Zeng, J. A. Bagnell, and M. Hebert, “Introspective perception: Learning to predict failures in vision systems,” in *2016 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pp. 1743–1750, IEEE, 2016.
- [4] J. K. Suhr and H. G. Jung, “Sensor fusion-based vacant parking slot detection and tracking,” *IEEE transactions on intelligent transportation systems*, vol. 15, no. 1, pp. 21–36, 2013.
- [5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” 2016.
- [6] E. Yurtsever, J. Lambert, A. Carballo, and K. Takeda, “A survey of autonomous driving: Common practices and emerging technologies,” *IEEE access*, vol. 8, pp. 58443–58469, 2020.
- [7] W. Li, M. Liao, G. Hua, Y. Zhang, and W. Zou, “Contextual guidance network for real-time semantic segmentation of autonomous driving,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, no. 10, pp. 16188–16203, 2025.

- [8] J. Zhao, Y. Wu, R. Deng, S. Xu, J. Gao, and A. Burke, “A survey of autonomous driving from a deep learning perspective,” *ACM Computing Surveys*, vol. 57, no. 10, pp. 1–60, 2025.
- [9] M. Ganesan, B. Chokkalingam, S. Kandhasamy, R. Verma, and L. Mihet-Popa, “Bridging the sim-to-real gap in motion planning for autonomous electric vehicles using autoware: A comparative study of simulation and real-world deployment,” *IEEE Access*, vol. 13, pp. 136210–136225, 2025.
- [10] M. V. L. De Carvalho, R. Simoni, L. R. Yoshioka, J. F. Justo, and B. M. K. Kawakami, “A performance evaluation of open source autonomous driving frameworks: Case studies of apollo and autoware,” *IEEE Access*, vol. 13, pp. 44294–44316, 2025.